MISE À DISPOSITION DE SERVICES

SIO 2025

Présenté par
Léo ADAM

Table des matières:

Introduction: Espace de travail
- 1) Organisation:
- 2) Logiciel nécessaire:

I: Les applications déployées sur Docker
- 1) Qu'est ce que docker?
- 2) Les applications déployées

II) Docker Compose
- Qu'est-ce que Docker Compose ?
- 1) Les parties importantes d'un docker compose
  - Précision sur la raisons du choix des ports:
- 2) Dockers composes des applications déployées
  - a) PostgreSQL:
  - b) MySQL:
  - c) PIA:
  - d) Portainer:
  - e) Baserow:
  - f) Projeqtor:
  - g) Wordpress:
  - h) Wordpress 2:
  - i) N8n:

III) Cartographie réseau et de ses flux de communication

Introduction: Espace de travail

1) Organisation:

J'ai commencé par structurer mes idées et répertorier les tâches à réaliser à l'aide de ClickUp. Il s'agit d'une plateforme dédiée à la planification et à la gestion de l'ensemble des aspects organisationnels d'un projet. Vous trouverez ci-dessous les liens vers le projet afin de pouvoir suivre l'avancement des différentes tâches.

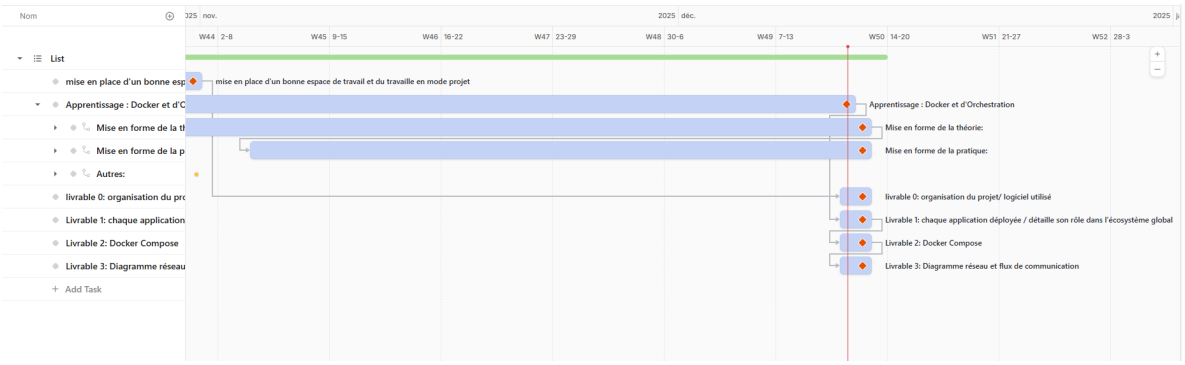
ClickUp

ClickUp is the highest-rated and fastest growing Productivity Platform.

https://app.clickup.com/90152097871/v/l/li/901518740625

Un résumé ce trouve ci-dessous:

Nom	Date d'échéance	Priorité
mise en place d'un bonne espace de travail et du tr...	10/31/25	
Apprentissage : Docker et d'Orchestration 3	Aujourd'hui	
Mise en forme de la théorie: 5	Demain, 4am	
Décrire les solutions techniques mises en place.		
documentation d'intégration précise.		
faire un récap explicatif de docker		Urgente
Documenter l'architecture applicative et résea...		
Assurer la reproductibilité et la maintenabilité...		
Mise en forme de la pratique: 6	Demain, 4:45am	
lister les app/ports de tout les containers		
Quelles applications ont été déployées et que...		
L'architecture logicielle : Comment les différe...		
L'intégration technique : Comment l'environn...		
L'architecture réseau : Quels sont les flux, inte...		
voir si toutes les app fonctionnent ensemble		Élevée
Autres: 3		
mettre a jour mon site wordpress		Basse
mettre a jour la map réseau	Demain, 4:45pm	Urgente
faire un fichier qui indique les ports utilisés		Normale
livrable 0: organisation du projet/ logiciel utilisé	Demain, 5:30pm	Urgente
Livrable 1: chaque application déployée / détaill...	Demain, 5:30pm	Urgente
Livrable 2: Docker Compose	Demain, 5:30pm	Urgente
Livrable 3: Diagramme réseau et flux de commu...	Demain, 5:30pm	Urgente
+ Ajouter Tâche		



2) Logiciel nécessaire:

Pour la réalisation du livrable, j'utilise les logiciels suivants:

- Notion: Permet de faire la rédaction, les tableau, la mise en page du livrable.
- Obsidian (avec l'extension Excalidraw): Permet de réaliser Les diagrammes.

Pour la mise en place des applications Docker:

- Visual Studio code: Permet d'éditer du code.
- Docker desktop: Permet de réaliser la gestion des containers.

I: Les applications déployées sur Docker

1) Qu'est ce que docker?

Docker est une technologie pour exécuter des applications dans des **conteneurs isolés**. Un conteneur est une unité standardisée de logiciel qui pack votre code et toutes ses dépendances, permettant à votre application de fonctionner rapidement et de manière fiable d'un environnement informatique à l'autre. Imaginez un colis hermétique pour votre application.
Docker est une plateforme qui permet d'empaqueter et d'exécuter des applications dans des **conteneurs** isolés.

Le concept clé : les conteneurs

Un conteneur est comme une boîte légère qui contient :

- Votre application
- Toutes ses dépendances (bibliothèques, outils, etc.)
- Un mini système d'exploitation

Pourquoi c'est utile ?

Le problème classique : "Ça marche sur mon ordinateur, mais pas sur le serveur !"

La solution Docker : L'application fonctionne partout de la même manière, car elle embarque tout son environnement avec elle.

Fonctionnement simplifié

1. **Image Docker** : C'est le modèle, la recette de votre application (comme un template)
2. **Conteneur** : C'est une instance en cours d'exécution de cette image (comme un processus actif)

3. **Dockerfile** : Le fichier de configuration qui décrit comment construire l'image

2) Les applications déployées

Nom	Version	Objectif principal du service	Lien app/base de données	Nature de l'inter connexion	port utilisé
PostgreSQL	16	Base de données.	- Adminer - Baserow - Appflowy - n8n	————	5432:5432
MySQL	Latest à date de création	Base de données.	- Phpmyadmin - Projeqtor - Wordpress 1 - Wordpress 2	————	3306
Adminer	Latest à date de création	Système de gestion de base de données pour PostgreSQL.	- PostgreSQL	Connection TCP sur le port 5432	8081:80
Phpmyadmin	Latest à date de création	Système de gestion de base de données pour MySQL.	- MySQL	Connection TCP sur le port 3306	8080:80
PIA front	Latest à date de création	Outil de la CNIL pour réaliser des analyses d'impact sur la protection des données (AIPD/DPIA). Aide à évaluer et documenter les risques liés au traitement de données personnelles conformément au RGPD.	- PIA back	Connection TCP sur le port 3000	8088:80
PIA back	Latest à date de création	—————	- PostgreSQL PIA	Connection TCP sur le port 5432	3000:3000
PostgreSQL uniquement pour PIA	Latest à date de création	Base de données pour PIA.	- PIA back	————	————

Nom	Version	Objectif principal du service	Lien app/base de données	Nature de l'inter connexion	port utilisé
Portainer	Latest à date de création	Interface graphique de gestion pour Docker. Permet de superviser, déployer et gérer vos conteneurs, images et volumes Docker facilement sans ligne de commande.	Pas de lien à une base de données	————	8000:8000 / 9000:9000 / 9443:9443
Baserow	1.35.3	Base de données collaborative no-code permettant de créer des tableaux, formulaires et automatisations sans compétences techniques.	- PostgreSQL:	Connection TCP sur le port 5432	7759:443 / 86:80
Projeqtor	12.2.3-1	Outil de gestion de projets collaboratif permettant d'organiser les tâches, suivre l'avancement et coordonner les équipes (planification, kanban, gantt).	- MySQL	Connection TCP sur le port 3306	8096:80
Wordpress	Latest à date de création	CMS (système de gestion de contenu) le plus populaire au monde. Permet de créer et gérer des sites web, blogs, boutiques en ligne sans coder.	- MySQL	Connection TCP sur le port 3306	99:80

Nom	Version	Objectif principal du service	Lien app/base de données	Nature de l'inter connexion	port utilisé
Wordpress 2	Latest à date de création	CMS (système de gestion de contenu) le plus populaire au monde. Permet de créer et gérer des sites web, blogs, boutiques en ligne sans coder.	- MySQL	Connection TCP sur le port 3306	89:80
Appflowy	Latest à date de création	Alternative open-source à Notion. Espace de travail tout-en-un pour prendre des notes, gérer des projets, créer des bases de données et collaborer.	- PostgreSQL	Connection TCP sur le port 5432	80:80 / 443:443
N8n	Latest à date de création	Plateforme d'automatisation de workflows (comme Zapier). Permet de connecter différentes applications et automatiser des tâches répétitives sans/avec code.	- PostgreSQL	Connection TCP sur le port 5432	5678:5678

II) Docker Compose

Qu'est-ce que Docker Compose ?

Docker Compose est un outil qui permet de définir et gérer plusieurs conteneurs Docker en même temps via un seul fichier de configuration YAML (`docker-compose.yml`).
Au lieu de lancer chaque conteneur individuellement avec de longues commandes, vous décrivez toute votre architecture applicative dans un fichier, et vous lancez tout d'un coup avec `docker-compose up`

1) Les parties importantes d'un docker compose

Élément	Syntaxe/Exemple	Description et utilité
version	`version: '3.8'`	Indique la version de la syntaxe Docker Compose utilisée.
services	`services:` `web:` `db:`	Section principale contenant tous les conteneurs à déployer. Chaque service = un conteneur.
image	`image: postgres:15`	Spécifie l'image Docker à télécharger et utiliser depuis Docker Hub ou un registry.
build	`build: ./mon-app`	Alternative à `image` : construit une image à partir d'un Dockerfile situé dans le dossier spécifié.
ports	`ports:` `- "8080:80"`	**Mapping de ports** entre l'hôte et le conteneur.Format : `port_hôte:port_conteneur` Rend le service accessible depuis l'extérieur.
environment	`environment:` `POSTGRES_USER: admin` `POSTGRES_PASSWORD: secret`	Définit les **variables d'environnement** pour configurer le conteneur (credentials, chemins, paramètres).
volumes	`volumes:` `- db-data:/var/lib/postgresql/data` `- ./config:/etc/config`	**Persistence des données** :- Volumes nommés (gérés par Docker)- Montage de dossiers locauxSans volumes, les données disparaissent à l'arrêt !
networks	`networks:` `- frontend` `- backend`	Définit les **réseaux virtuels** pour organiser et isoler la communication entre conteneurs.
depends_on	`depends_on:` `- db`	Indique l'**ordre de démarrage** des services.⚠️ Ne garantit pas que le service soit prêt, juste qu'il a démarré.
restart	`restart: always`	**Politique de redémarrage automatique** :- `no` : jamais- `always` : toujours- `on-failure` : en cas d'erreur- `unless-stopped` : sauf arrêt manuel

Précision sur la raisons du choix des ports:

Le mappage des ports se fait sous le format `hôte:conteneur.`

Exemple 8080:80

- Docker intercepte cette requête sur le port 8080 de votre machine
- Docker redirige automatiquement vers le port 80 **à l'intérieur** du conteneur

Critère	Explication
Éviter les conflits	Si le port 80 est déjà utilisé sur votre machine, impossible d'utiliser `80:80`
Plusieurs instances	Lancer plusieurs versions du même service simultanément

Critère	Explication
Conventions et standards	Faciliter la mémorisation et respecter les usages
Sécurité	Masquer les ports standards pour éviter les scans automatiques
Organisation multi-projets	Séparer les projets par plages de ports
Environnements différents	Différencier dev, staging, production

2) Dockers composes des applications déployées

a) PostgreSQL:

```yaml
services:
  postgresdb:
    container_name: postgresdb
    build: .
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: 'Password1'
      POSTGRES_DB: 'postgres_production'
      POSTGRES_USER: 'postgresuser'
    volumes:
      - ./postgresdb:/var/lib/postgresql/data
    networks:
      - network_postgres1
    ports:
      - "5432:5432"
  adminer:
    image: adminer
    ports:
      - 8081:8080
    networks:
      - network_postgres1
volumes:
  postgresdb: {}
networks:
  network_postgres1:
    name: postgres1 # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Élément	Description	Utilité
`image: postgresdb`	Base de données PostgreSQL	Stockage relationnel
`container_name: postgresdb`	Nom du conteneur	Identification unique
`build: .`	Construction personnalisée	Utilise un Dockerfile local
`POSTGRES_PASSWORD: 'Password1'`	Mot de passe root	Authentification admin
`POSTGRES_DB: 'postgres_production'`	Base par défaut	Base de données créée au démarrage
`POSTGRES_USER: 'postgresuser'`	Utilisateur principal	Compte avec tous les droits
`volumes: ./postgresdb:/var/lib/postgresql/data`	Persistance des données	Sauvegarde de la base
`ports: "5432:5432"`	Port PostgreSQL	Accès direct à la base
`image: adminer`	Interface web DB	Gestion légère de bases de données
`ports: "8081:8080"`	Interface Adminer	Accès via http://localhost:8081
`network_postgres1`	Réseau partagé	Communication entre services

b) MySQL:

```yaml
version: "3"
services:
  db:
    image: mysql:latest
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: Password1
    networks:
      - frontend
      - frontend2
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: rootpassword
    networks:
      - frontend
volumes:
  mysql: {}
networks:
  frontend:
    name: network-wordpress # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
  frontend2:
    name: network-wordpress2 # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Élément	Description	Utilité
`image: mysql:latest`	Base de données MySQL	Stockage des données WordPress
`MYSQL_ROOT_PASSWORD: rootpassword`	Mot de passe root	Accès administrateur complet
`MYSQL_DATABASE: wordpress`	Nom de la base	Base dédiée à WordPress
`MYSQL_USER: wpuser`	Utilisateur applicatif	Compte pour WordPress (moins de privilèges)
`networks: frontend/frontend2`	Réseaux multiples	Permet la communication avec plusieurs services
`phpmyadmin`	Interface web MySQL	Gestion visuelle de la base de données
`ports: "8080:80"`	Accès phpMyAdmin	Interface accessible via http://localhost:8080
`PMA_HOST: db`	Connexion à MySQL	Liaison phpMyAdmin-MySQL

c) PIA:

```yaml
version: '3'
services:
  cnil-pia-back:
    image: lrqdo/cnil-pia-back
    restart: unless-stopped
    environment:
      DATABASE_HOST: 'database'
      DATABASE_USERNAME: 'postgres'
      DATABASE_PASSWORD: 'password1'
      SECRET_KEY_BASE: 'somesecret'
    links:
      - database
    depends_on:
      - database
    ports:
      - 3000:3000

  cnil-pia-front:
    image: lrqdo/cnil-pia-front
    restart: unless-stopped
    ports:
      - 0.0.0.0:8088:80

  database:
    restart: unless-stopped
    image: postgres:10.1
    environment:
      POSTGRES_PASSWORD: 'password1'
```

Élément	Description	Utilité
`image: lrqdo/cnil-pia-back`	Backend PIA	API pour l'analyse d'impact RGPD
`image: lrqdo/cnil-pia-front`	Frontend PIA	Interface utilisateur web
`DATABASE_HOST: 'database'`	Hôte PostgreSQL	Connexion à la base de données
`DATABASE_USERNAME: 'postgres'`	Utilisateur DB	Authentification PostgreSQL
`SECRET_KEY_BASE: 'somesecret'`	Clé de chiffrement	Sécurisation des sessions
`links: - database`	Liaison de services	Connexion directe entre conteneurs
`depends_on: - database`	Dépendance de démarrage	Assure que la DB démarre avant l'app
`ports: "3000:3000"` (backend)	API REST	Accès aux services backend

Élément	Description	Utilité
`ports: "0.0.0.0:8088:80"` (frontend)	Interface web	Accès public à l'application
`image: postgres:10.1`	Base de données	Stockage des données PIA

---

d) Portainer:

```yaml
services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: unless-stopped
    ports:
      - "9000:9000"   # Web UI access (HTTP)
      - "9443:9443"   # Web UI access (HTTPS)
      - "8000:8000"   # Edge Agent port
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - portainer_network

  agent:
    image: portainer/agent:latest
    container_name: portainer_agent
    restart: unless-stopped
    environment:
      AGENT_CLUSTER_ADDR: tasks.agent
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /var/lib/docker/volumes:/var/lib/docker/volumes
    networks:
      - portainer_network
networks:
  portainer_network:
volumes:
  portainer_data:
```

Élément	Description	Utilité
`image: portainer/portainer-ce:latest`	Interface de gestion Docker	GUI pour gérer les conteneurs
`ports: "9000:9000"`	Interface web HTTP	Accès principal à Portainer
`ports: "9443:9443"`	Interface web HTTPS	Accès sécurisé
`ports: "8000:8000"`	Edge Agent	Port pour les agents distants
`volumes: /var/run/docker.sock`	Socket Docker	Communication avec le daemon Docker
`volumes: portainer_data:/data`	Données Portainer	Persistance de la configuration
`image: portainer/agent:latest`	Agent Portainer	Gestion de l'hôte Docker
`container_name: portainer_agent`	Nom de l'agent	Identification de l'agent
`AGENT_CLUSTER_ADDR: tasks.agent`	Adresse cluster	Mode Swarm (clustering)
`volumes: /var/lib/docker/volumes`	Volumes Docker	Accès aux volumes pour inspection
`portainer_network`	Réseau dédié	Communication Portainer-Agent

---

e) Baserow:

```yaml
services:
  baserow:
    container_name: baserow
    image: baserow/baserow:1.35.3
    environment:
      BASEROW_PUBLIC_URL: http://localhost:86
      DATABASE_URL: postgresql://postgresuser:Password1@postgresdb:5432/baserowdb
    ports:
      - "86:80"
      - "7759:443"
    volumes:
      - baserow_data:/baserow/data
    networks:
      - network_postgres1
volumes:
  baserow_data:
networks:
  network_postgres1:
    name: postgres1 # Use a custom name
    external: true
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Élément	Description	Utilité
`image: baserow/baserow:1.35.3`	Plateforme no-code	Alternative open-source à Airtable
`BASEROW_PUBLIC_URL: http://localhost:86`	URL publique	Accès à l'interface Baserow
`DATABASE_URL: postgresql://...`	Connexion PostgreSQL	Lien vers la base de données externe
`ports: "86:80", "7759:443"`	HTTP et HTTPS	Accès web sécurisé et non sécurisé
`volumes: baserow_data:/baserow/data`	Stockage persistant	Sauvegarde des données Baserow
`network_postgres1`	Réseau personnalisé	Communication avec PostgreSQL
`external: true`	Réseau externe	Utilise un réseau créé séparément
`driver_opts: com.docker.network.bridge.host_binding_ipv4: 127.0.0.1`	Binding local	Limite l'accès à localhost uniquement

f) Projeqtor:

```yaml
services:
  projeqtor:
    image: mraber/projeqtor:12.2.3-1
    restart: unless-stopped
    environment:
      PJQ_PHP_MAX_INPUT_VARS: 4000
      PJQ_PHP_REQUEST_TERMINATE_TIMEOUT: 0
      PJQ_PHP_MAX_EXECUTION_TIME: 30
      PJQ_PHP_MEMORY_LIMIT: "512M"
      PJQ_PHP_MAX_UPLOAD_SIZE: "512M"
      PJQ_DB_TYPE: "mysql"
      PJQ_DB_HOST: "db:3306"
      PJQ_DB_NAME: "projeqtor"
      PJQ_DB_USER: "wpuser"
      PJQ_DB_PASSWORD: "Password1"
      PJQ_DB_DISPLAY_NAME: "Projeqtor"
      PJQ_DB_COLLATION: "utf8mb4_general_ci"
      PJQ_DIR_ATTACHMENTS: "/var/data/projeqtor/attachments"
      PJQ_DIR_DOCUMENTS: "/var/data/projeqtor/documents"
      PJQ_LOG_FILE: "/var/data/projeqtor/logs/projeqtor_$${date}.log"
      # Levels : debug (4) -> info (2)
      PJQ_LOG_LEVEL: 2
    volumes:
      - prj_attachments:/var/data/projeqtor/attachments
      - prj_documents:/var/data/projeqtor/documents
      - prj_logs:/var/data/projeqtor/logs
    ports:
      - 8096:80
    logging:
      # Do not fill disk with logs!
      driver: "json-file"
      options:
        max-size: "10M"
        max-file: "5"
        compress: "true"
    networks:
      - frontend
volumes:
  prj_db_data: {}
  prj_attachments: {}
  prj_documents: {}
  prj_logs: {}
networks:
  frontend:
    name: network-wordpress # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Élément	Description	Utilité
`image: mnaber/projector: 12.2.3-1`	Application Projector	Gestionnaire de projets (type Jira/Trello)
`PJO_PHP_MAX_ INPUT_VARS: 4000`	Variables PHP	Limite de données POST
`PJO_PHP_REQUEST _TERMINATE_TIMEOUT: 0`	Timeout requêtes	Pas de limite de temps
`PJO_PHP_MAX_ EXECUTION_TIME: 30`	Temps d'exécution	Maximum 30 secondes
`PJO_PHP_MEMORY_LIMIT: "512M"`	Mémoire PHP	512 Mo alloués
`PJO_PHP_MAX_ UPLOAD_SIZE: "512M"`	Taille upload	Fichiers jusqu'à 512 Mo
`PJO_DB_TYPE: "mysql"`	Type de base	Utilise MySQL
`PJO_DB_HOST: "db:3306"`	Hôte MySQL	Connexion à la base
`PJO_DB_NAME: "projector"`	Nom de la base	Base dédiée à Projector
`PJO_DB_USER: "wpuser"`	Utilisateur DB	Authentification MySQL
`PJO_DB_DISPLAY_NAME: "Projector"`	Nom d'affichage	Libellé dans l'interface
`PJO_DB_COLLATION: "utf8mb4_general_ci"`	Collation	Support Unicode complet
`PJO_DIR_ATTACHMENTS: "/var/data/projector/ attachments"`	Dossier pièces jointes	Stockage des fichiers uploadés
`PJO_DIR_DOCUMENTS: "/var/data/projector/ documents"`	Dossier documents	Stockage des documents
`PJO_LOG_FILE: "/var/data /projector/logs/ projector_${date}.log"`	Fichiers de logs	Journalisation avec date
`PJO_LOG_LEVEL: 2`	Niveau de log	Info (détail moyen)
`logging: driver: "json-file"`	Format des logs	Logs en JSON
`logging: max-size: "10M"`	Taille maximale	10 Mo par fichier de log
`logging: max-file: "5"`	Rotation des logs	Garde 5 fichiers maximum
`logging: compress: "true"`	Compression	Économise l'espace disque

Élément	Description	Utilité
`volumes: prj_attachments: {}`	Volume vide	Créé automatiquement
`ports: "8096:80"`	Interface web	Accès via http://localhost:8096

g) Wordpress:

```yaml
services:
  wordpress-new:
    image: wordpress
    container_name: "wordpress1"
    restart: unless-stopped
    ports:
      - "99:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: "wpuser"
      WORDPRESS_DB_PASSWORD: "Password1"
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - ./wordpress:/var/www/html
      - /etc/localtime:/etc/localtime:ro
    networks:
      - frontend
networks:
  frontend:
    name: network-wordpress # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
    external: true
```

Élément	Description	Utilité
`image: wordpress`	Image officielle WordPress	CMS pour créer et gérer un site web
`container_name: wordpress1/ wordpress2`	Nom unique du conteneur	Identification et gestion du conteneur
`restart: unless-stopped`	Politique de redémarrage	Assure la disponibilité du service
`ports: "99:80"`	Mappage de ports	Accès au WordPress via http://localhost:99
`WORDPRESS_DB_HOST: db`	Hôte de la base de données	Connexion au conteneur MySQL
`WORDPRESS_DB_USER: wpuser`	Utilisateur de la base	Authentification MySQL
`WORDPRESS_DB_ PASSWORD:Password1`	Mot de passe DB	Sécurisation de l'accès
`volumes: ./wordpress: /var/www/html`	Persistance des données	Sauvegarde des fichiers WordPress
`volumes: /etc/localtime:/etc /localtime:ro`	Synchronisation de l'heure	Alignement du fuseau horaire
`network-wordpress`	Réseau personnalisé	Isolation et communication inter-conteneurs

h) Wordpress 2:

```yaml
services:
  wordpress-new:
    image: wordpress
    container_name: "wordpress2"
    restart: unless-stopped
    ports:
      - "89:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: "wpuser"
      WORDPRESS_DB_PASSWORD: "Password1"
      WORDPRESS_DB_NAME: wordpress2
    volumes:
      - ./wordpress:/var/www/html
      - /etc/localtime:/etc/localtime:ro
    networks:
      - frontend2
networks:
  frontend2:
    name: network-wordpress2 # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

---

i) N8n:

```yaml
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    container_name: n8n
    security_opt:
      - no-new-privileges:true
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      N8N_HOST: n8n.it-connect.local
      N8N_PORT: 5678
      N8N_PROTOCOL: https
      N8N_SECURE_COOKIE: false
      N8N_RUNNERS_ENABLED: true
      N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS: true
      NODE_ENV: production
      WEBHOOK_URL: https://n8n.it-connect.local
      WEBHOOK_TUNNEL_URL: https://n8n.it-connect.local
      GENERIC_TIMEZONE: Europe/Paris
      TZ: Europe/Paris
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: postgresdb
      DB_POSTGRESDB_DATABASE: n8n
      DB_POSTGRESDB_PORT: 5432
      DB_POSTGRESDB_USER: postgresuser
      DB_POSTGRESDB_PASSWORD: Password1
    volumes:
      - n8n_data:/home/node/.n8n:rw
      - n8n_files:/files:rw
    networks:
      - network_postgres1
volumes:
  n8n_data: {}
  n8n_files: {}
networks:
  network_postgres1:
    name: postgres1 # Use a custom name
    external: true
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Élément	Description	Utilité
`image: docker.n8n.io/n8nio/n8n`	Plateforme d'automatisation	Alternative open-source à Zapier
`container_name: n8n`	Nom du conteneur	Identification unique
`security_opt: no-new-privileges:true`	Sécurité renforcée	Empêche l'escalade de privilèges
`N8N_HOST: n8n.it-connect.local`	Nom d'hôte	Accès via nom de domaine
`N8N_PORT: 5678`	Port interne	Port d'écoute de n8n
`N8N_PROTOCOL: https`	Protocole sécurisé	Communication chiffrée
`N8N_SECURE_COOKIE: false`	Configuration cookie	Désactive les cookies sécurisés (dev)
`N8N_RUNNERS_ENABLED: true`	Workers d'exécution	Active les tâches asynchrones

Élément	Description	Utilité
`N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS: true`	Permissions strictes	Sécurise les fichiers de config
`NODE_ENV: production`	Environnement	Mode production optimisé
`WEBHOOK_URL: https://n8n.it-connect.local`	URL webhooks	Point d'entrée pour les déclencheurs
`WEBHOOK_TUNNEL_URL: https://...`	Tunnel public	Accès externe via tunnel
`GENERIC_TIMEZONE: Europe/Paris`	Fuseau horaire	Planification correcte des tâches
`TZ: Europe/Paris`	Timezone système	Synchronisation de l'heure
`DB_TYPE: postgresdb`	Type de base	Utilise PostgreSQL
`DB_POSTGRESDB_HOST: postgresdb`	Hôte de la base	Connexion PostgreSQL
`volumes: n8n_data:/home/node/.n8n:rw`	Données persistantes	Workflows et configurations
`volumes: n8n_files:/files:rw`	Fichiers utilisateur	Stockage des fichiers traités

III) Cartographie réseau et de ses flux de communication

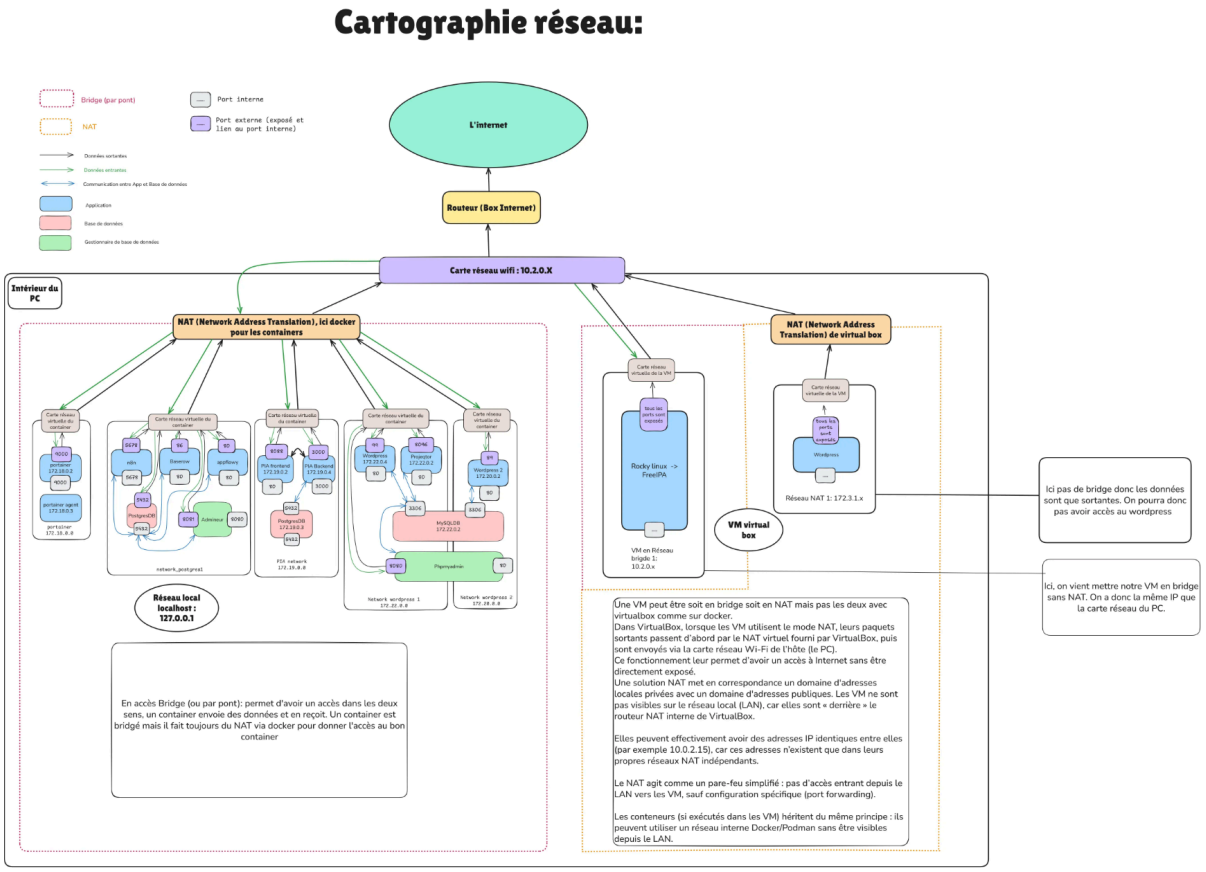
Cette infrastructure Docker héberge un écosystème complet de services web et d'outils métier conteneurisés, permettant :

✅ Hébergement de sites WordPress (production et développement)
✅ Gestion de projets avec Projector
✅ Conformité RGPD via l'outil PIA de la CNIL
✅ Automatisation de workflows avec n8n
✅ Gestion de bases de données no-code avec Baserow
✅ Administration centralisée via Portainer

AVANTAGES DE L'ARCHITECTURE :

- Isolation : Chaque service fonctionne dans un environnement isolé
- Portabilité : Infrastructure reproductible sur n'importe quel hôte Docker
- Scalabilité : Ajout de services sans impact sur l'existant

- Sécurité : Segmentation réseau et exposition contrôlée des ports
- Maintenance : Gestion simplifiée via Portainer



[cartographie réseau docker v2.pdf]